AMAL JYOTHI COLLEGE OF ENGINEERING KANJIRAPPALLY

B. TECH - COMPUTER SCIENCE AND ENGINEERING
BTCS2023-27-S6: 23CST362-PROGRAMMING IN PYTHON-Project (Batch – I)

Modern Marks Calculator GUI

Topic Title	Modern Marks Calculator GUI
Student Name	Noel B Augustine
Register Number	AJC23CS163
Class/Section	CSE C

Abstract

Calculating student grades manually can be time-consuming and prone to human error. This project implements a modern, automated desktop application that calculates a student's total marks, percentage, and corresponding letter grade based on their input scores. To meet the requirement of using a library beyond the core syllabus, the application utilizes customtkinter, which provides a contemporary, dark-mode-compatible graphical user interface (GUI) that significantly improves upon standard Tkinter. The application features built-in input validation to prevent crashes from non-numeric or out-of-bounds data. Ultimately, this project demonstrates a complete, simple, and stable practical application of Python GUI development and logic implementation.

Problem Description

Educational institutions and individuals frequently need to calculate aggregate totals, percentages, and categorical letter grades from raw test scores. Performing these calculations manually is inefficient. The objective of this project is to develop a Python-based interactive desktop application that streamlines this process. The system must accept inputs for three subjects (Math, Science, and English), mathematically process these inputs to generate a total and percentage, and apply conditional logic to determine a final grade. Furthermore, the application must handle user errors gracefully, ensuring that invalid inputs (like text or scores exceeding the maximum limit) do not cause runtime failures.

Libraries & Tools Used (with purpose)

- CustomTkinter (Beyond Syllabus Library): Used to construct the modern Graphical User Interface. It acts as a wrapper around standard Tkinter, providing advanced UI elements like rounded frames, dynamic color themes, and system-aware light/dark modes that are not natively available in standard Python.

- Tkinter.messagebox (Standard Library): Used to generate pop-up dialogue boxes for error handling, specifically warning the user when invalid data is entered into the calculator.

System Design / Workflow

1. Input Phase: The user interface renders input fields for three subjects. The system waits for the user to trigger the "Calculate Results" button.
2. Validation Phase: The system reads the string inputs, attempts to convert them to floating-point numbers, and checks if they fall within the logical 0–100 range.
3. Processing Phase:

- Calculates the sum of the three subjects.
- Calculates the percentage based on a maximum score of 300.
- Evaluates the percentage against a predefined grading scale (A to F) using conditional statements.

4. Output Phase: The GUI updates dynamically to display the calculated total, percentage, and a color-coded letter grade.

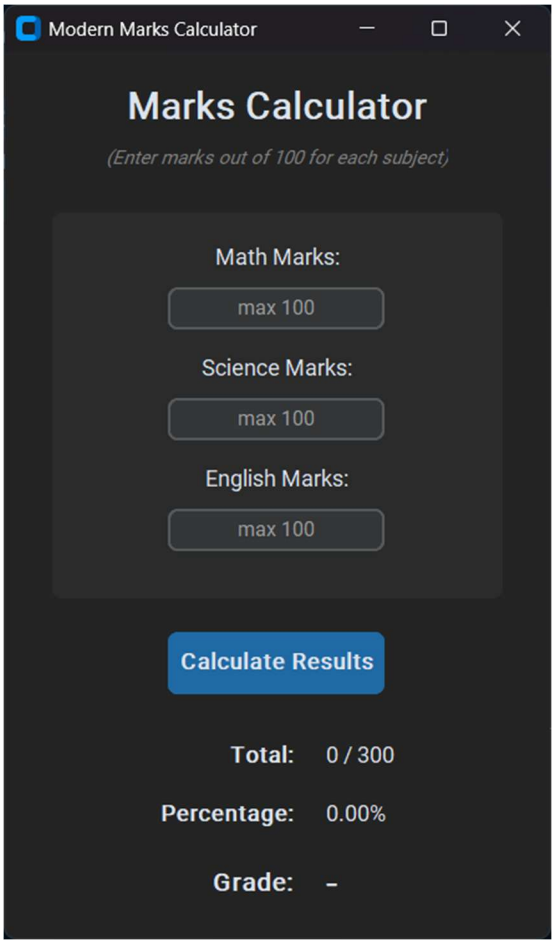
Implementation Details

The application is built entirely in Python. The UI is initialized by establishing a main ctk.CTk() window. Input fields are grouped within a CTkFrame to create a modern "card" layout, utilizing CTkEntry widgets configured with placeholder text.

The core processing logic is housed within the calculate_results() function. Data is fetched using the .get() method. A try-except ValueError block acts as a safety net to catch string inputs. An if statement utilizes the any() function to iterate through the inputs and verify that no score is less than 0 or greater than 100. The grading logic uses an if-elif-else chain to assign both a string value (the grade) and a hex color code. Finally, the .configure() method is called on the output CTkLabel widgets to render the final calculations back to the user interface.

Results & Output (with screenshots)

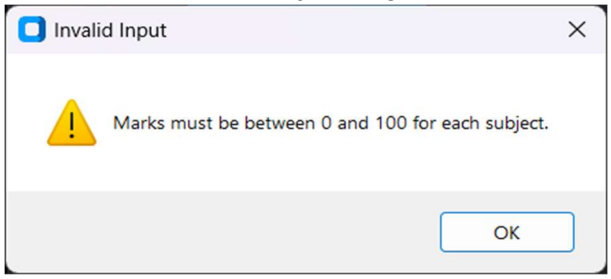
- The application successfully calculates and displays grades while mitigating user input errors.
- Screenshot 1: Input / Interface / Start screen



- Description: The initial modern interface awaiting user input.

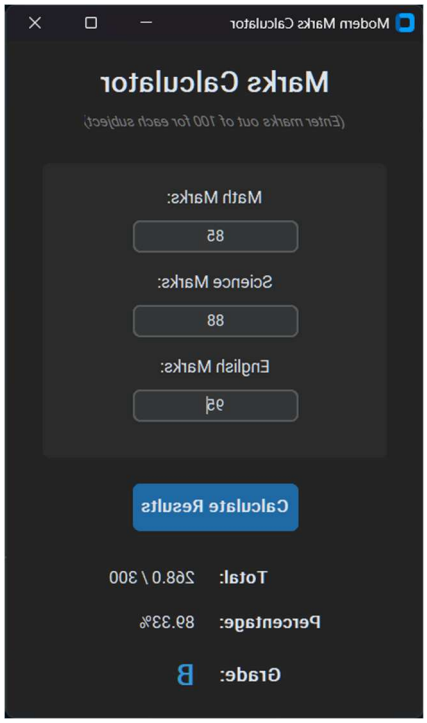
- Screenshot 2: Core processing or intermediate output

When mark of one subject is greater than 100 it shows up the following error



- Description: The validation logic intercepting an out-of-bounds input to prevent calculation errors.

- Screenshot 3: Final output / result



- Description: The final processed output displaying the total, percentage, and dynamic letter grade.

Conclusion & Scope for Mini Project Extension

This project successfully demonstrates a stable, standalone implementation of a modern Python GUI using customtkinter. The program fulfills all mathematical processing and validation requirements without runtime errors.

Scope for Extension:

- This individual mini-project can easily serve as a foundational module for a larger Mini Project. The current logic can be extended into a comprehensive "Student Management System." Future iterations could integrate an SQLite database to permanently store the calculated records, add data visualization (like matplotlib) to track student performance over time, or allow batch processing via CSV file uploads. This ensures continuity in problem understanding and design for future development phases.

References

5. Python Software Foundation. (n.d.). Python Language Reference. Retrieved from python.org
6. Schimansky, T. (n.d.). CustomTkinter Documentation. Retrieved from github.com/TomSchimansky/CustomTkinter

Appendix: Source Code

```
import customtkinter as ctk
from tkinter import messagebox

ctk.set_appearance_mode("System")
ctk.set_default_color_theme("blue")

def calculate_results():
    try:
        # Fetching Inputs
        math_score = float(entry_math.get())
        science_score = float(entry_science.get())
        english_score = float(entry_english.get())

        # --- NEW: Validation Logic ---
        # Check if any score is out of the 0-100 bounds
        if any(score < 0 or score > 100 for score in [math_score,
science_score, english_score]):
            messagebox.showwarning("Invalid Input", "Marks must be between 0
and 100 for each subject.")
            return

        # Mathematical Logic
        total = math_score + science_score + english_score
        max_marks = 300
        percentage = (total / max_marks) * 100

        # Grading and Color Logic
        if percentage >= 90:
            grade = "A"
            color = "#2ecc71"
        elif percentage >= 80:
            grade = "B"
            color = "#3498db"
        elif percentage >= 70:
            grade = "C"
            color = "#f1c40f"
        elif percentage >= 60:
            grade = "D"
            color = "#e67e22"
        else:
            grade = "F"
            color = "#e74c3c"

        # Updating the UI
        label_total_result.configure(text=f"{total:.1f} / {max_marks}")
        label_percentage_result.configure(text=f"{percentage:.2f}%")
        label_grade_result.configure(text=grade, text_color=color)

    except ValueError:
        messagebox.showerror("Input Error", "Please enter valid numbers for all
subjects.")

# --- Modern GUI Setup ---
```

```
root = ctk.CTk()
root.title("Modern Marks Calculator")
root.geometry("350x570")

# App Title
title_label = ctk.CTkLabel(root, text="Marks Calculator",
font=ctk.CTkFont(size=24, weight="bold"))
title_label.pack(pady=(20, 5))

# --- NEW: Helpful Info Label ---
info_label = ctk.CTkLabel(root, text="(Enter marks out of 100 for each
subject)", font=ctk.CTkFont(size=12, slant="italic"), text_color="gray")
info_label.pack(pady=(0, 10))

# --- Input Frame ---
input_frame = ctk.CTkFrame(root)
input_frame.pack(pady=10, padx=30, fill="both", expand=True)

# Math Input
ctk.CTkLabel(input_frame, text="Math Marks:",
font=ctk.CTkFont(size=14)).pack(pady=(15, 0))
entry_math = ctk.CTkEntry(input_frame, placeholder_text="max 100",
justify="center")
entry_math.pack(pady=(5, 10))

# Science Input
ctk.CTkLabel(input_frame, text="Science Marks:",
font=ctk.CTkFont(size=14)).pack(pady=(0, 0))
entry_science = ctk.CTkEntry(input_frame, placeholder_text="max 100",
justify="center")
entry_science.pack(pady=(5, 10))

# English Input
ctk.CTkLabel(input_frame, text="English Marks:",
font=ctk.CTkFont(size=14)).pack(pady=(0, 0))
entry_english = ctk.CTkEntry(input_frame, placeholder_text="max 100",
justify="center")
entry_english.pack(pady=(5, 15))

# --- Calculate Button ---
calc_button = ctk.CTkButton(root, text="Calculate Results",
command=calculate_results, font=ctk.CTkFont(size=14, weight="bold"), height=40)
calc_button.pack(pady=10)

# --- Results Frame ---
result_frame = ctk.CTkFrame(root, fg_color="transparent")
result_frame.pack(pady=10)

ctk.CTkLabel(result_frame, text="Total:", font=ctk.CTkFont(size=14,
weight="bold")).grid(row=0, column=0, padx=10, pady=5, sticky="e")
label_total_result = ctk.CTkLabel(result_frame, text="0 / 300",
font=ctk.CTkFont(size=14))
label_total_result.grid(row=0, column=1, padx=10, pady=5, sticky="w")

ctk.CTkLabel(result_frame, text="Percentage:", font=ctk.CTkFont(size=14,
```

```
weight="bold")).grid(row=1, column=0, padx=10, pady=5, sticky="e")
label_percentage_result = ctk.CTkLabel(result_frame, text="0.00%",
font=ctk.CTkFont(size=14))
label_percentage_result.grid(row=1, column=1, padx=10, pady=5, sticky="w")

ctk.CTkLabel(result_frame, text="Grade:", font=ctk.CTkFont(size=16,
weight="bold")).grid(row=2, column=0, padx=10, pady=10, sticky="e")
label_grade_result = ctk.CTkLabel(result_frame, text="-",
font=ctk.CTkFont(size=24, weight="bold"))
label_grade_result.grid(row=2, column=1, padx=10, pady=10, sticky="w")

# Run the application
root.mainloop()
```